PayStream: Design Document

Nov 2, 2026 · @Partha Anil

PayStream is a real-time payments fraud and analytics platform. It ingests UPI-style transactions through Kafka, catches fraud within seconds with Flink, builds an Iceberg lakehouse on S3 with Spark, models a star-schema warehouse in Snowflake with dbt, and serves analysts through a React console. Everything is built with free tools, runs locally first, and moves to AWS and Snowflake trials only in the final phase.

Goals and non-goals

PayStream must prove one thing to a hiring manager: the author can design, build, test, deploy and operate a production-style data platform end to end.

Goals

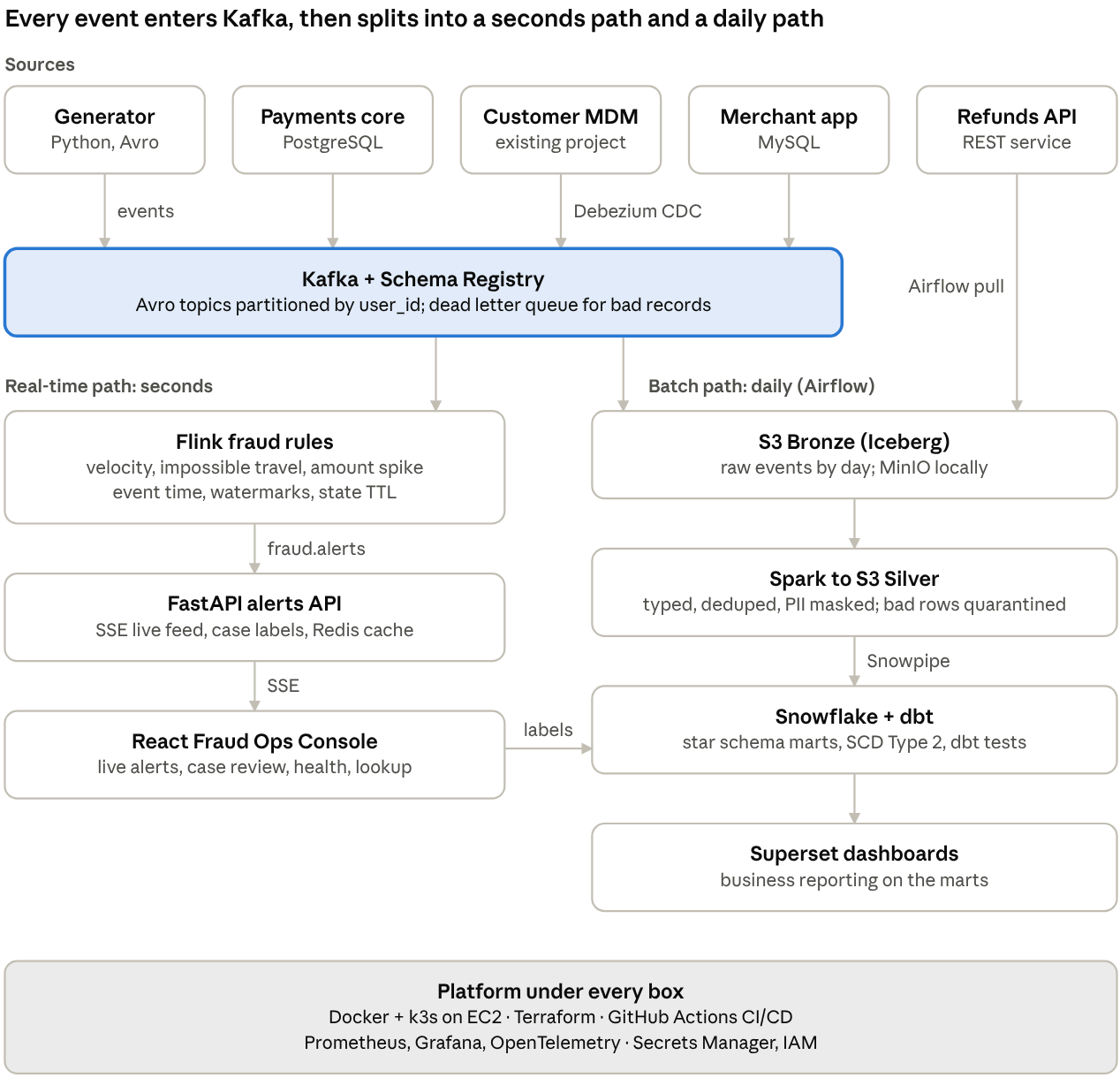
1. Process a continuous stream of payment events and flag fraud within seconds, with measured latency.
2. Keep a full, replayable history in an open lakehouse (Iceberg on S3) that can be backfilled and rolled back.
3. Serve business questions from a star-schema warehouse in Snowflake, modelled and tested with dbt.
4. Integrate data from real internal applications: the existing Customer Data Integration Pipeline, a Postgres payments core and a MySQL merchant app.
5. Validate data at every layer and quarantine bad records instead of dropping them.
6. Deploy everything as code: Docker, Kubernetes (k3s), Terraform and GitHub Actions.
7. Publish measured numbers in the README: throughput, end-to-end latency, rows processed, query speed-ups and daily cloud cost.
8. Give analysts a working React console to review alerts, which feeds labels back into the warehouse.

Non-goals

- Real payment data. Real data is private; a generator modelled on PaySim with injected fraud provides known ground truth instead.
- A production machine-learning model. Fraud detection uses stateful rules first; an ML scorer is an optional extension.

- Multi-region deployment, CDN, WAF, blue-green environments and managed EKS/MSK. These add cost without adding learning at this scale.
- Mobile apps or customer-facing payment screens.

Architecture at a glance



PayStream architecture · 5 sources, 2 paths, 1 platform

Kafka is the backbone. Flink reads it to raise fraud alerts within seconds, while the daily batch path lands every event in S3, cleans it with Spark and models it in Snowflake. Analyst labels from the console flow back into the warehouse.

Components

Every tool earns its place with a one-line reason; a tool without one is cut.

Layer	Tool	Job in PayStream	Why this tool
Source	Python transaction generator	Emits payments, users, merchants and injected fraud at a configurable rate	Real payment data is private; known ground truth lets us measure precision and recall
Source	PostgreSQL 16	Payments core (users, accounts, balances) and the Customer Data Integration Pipeline's golden records	Industry-standard OLTP database; the old project already runs on it
Source	MySQL 8	Merchant onboarding app	Real companies have mixed source engines; shows CDC from two engines
Source	Refunds REST API (FastAPI)	Internal service pulled in batch	Shows API ingestion with retries, backoff and a circuit breaker
Ingestion	Debezium	Streams Postgres and MySQL row changes (CDC) into Kafka	Reads the database log, so the source app needs no code changes
Ingestion	Apache Kafka + Schema Registry (Avro)	Durable event backbone; topics partitioned by `user_id`	Replayable, ordered per key, decouples producers from consumers
Real-time	Apache Flink	Stateful fraud rules, 1-minute merchant metrics, alerts topic	Event-time windows, watermarks and exactly-once sinks; fraud must be caught in seconds

Layer	Tool	Job in PayStream	Why this tool
Batch	Apache Spark	Bronze to Silver cleaning, dedupe, backfills, compaction	Large batch jobs over a year of history (about 100M rows)
Storage	Amazon S3 (MinIO locally) + Apache Iceberg	Lakehouse in Bronze and Silver layers	Open table format with partitions, schema evolution and time travel
Warehouse	Snowflake (AWS region)	Star-schema marts for analytics	High demand in data engineering roles; loads from S3 via Snowpipe; trial needs no card
Transformation	dbt Core	Staging, intermediate and mart models; tests and docs	SQL-first ELT with version control, tests and lineage
Orchestration	Apache Airflow	Schedules batch runs, loads, dbt, quality gates and backfills	Industry standard; retries, SLAs and backfills built in
Quality	Great Expectations + dbt tests	Validates every layer; failures block publishing	Declarative checks with readable reports
Serving	FastAPI	Alerts API, case review API, SSE live feed	Async Python, typed with Pydantic, auto-generated OpenAPI docs
Cache	Redis	User-profile lookups for the API; cleared by CDC events	Cuts repeat database reads
BI	Apache Superset	Business dashboards on Snowflake	Free, open source; no need to rebuild charts by hand

Layer	Tool	Job in PayStream	Why this tool
Frontend	React + TypeScript + Vite	Fraud Ops Console for analysts	Most-requested frontend stack; analysts create labels, not just view data
Platform	Docker, k3s, Helm	Packaging and running every service	Same images run locally (k3d) and on EC2
Infrastructure	Terraform	AWS (S3, IAM, EC2, ECR, Secrets Manager) and Snowflake (warehouses, roles, grants)	Repeatable environments; `terraform destroy` saves credits
CI/CD	GitHub Actions	Lint, tests, data checks, image builds, Terraform plan and deploy	Free for public repositories
Observability	Prometheus, Grafana, OpenTelemetry, Jaeger	Metrics, dashboards, alerts and traces	Open-source standard stack

Data flow

One event takes two paths: a real-time path that answers "is this fraud?" in seconds, and a batch path that answers business questions the next morning.

Kafka topics

Topic	Producer	Key	Consumers
`payments.transactions`	Generator	`user_id`	Flink, Bronze sink
`cdc.payments_core.*`	Debezium (Postgres)	Primary key	Flink (enrichment), Bronze sink, cache invalidator

Topic	Producer	Key	Consumers
`cdc.merchants.*`	Debezium (MySQL)	Primary key	Flink (enrichment), Bronze sink
`fraud.alerts`	Flink	`user_id`	Alerts API, Bronze sink
`payments.dlq`	Any consumer	Original key	Quarantine job, on-call review

Real-time path (seconds)

1. The generator publishes each payment as an Avro record; the Schema Registry rejects any record that breaks the schema.
2. Flink reads by event time with watermarks, so late events land in the right window.
3. Flink joins each payment with the latest user and merchant state from the CDC topics.
4. Stateful rules run per user: velocity (more than 5 payments in 1 minute), impossible travel (two cities too far apart for the time gap) and amount spike (10 times the user's running average). State has a TTL so memory cannot grow forever.
5. Matches go to `fraud.alerts`; the Alerts API pushes them to the console through Server-Sent Events.

Batch path (daily, orchestrated by Airflow)

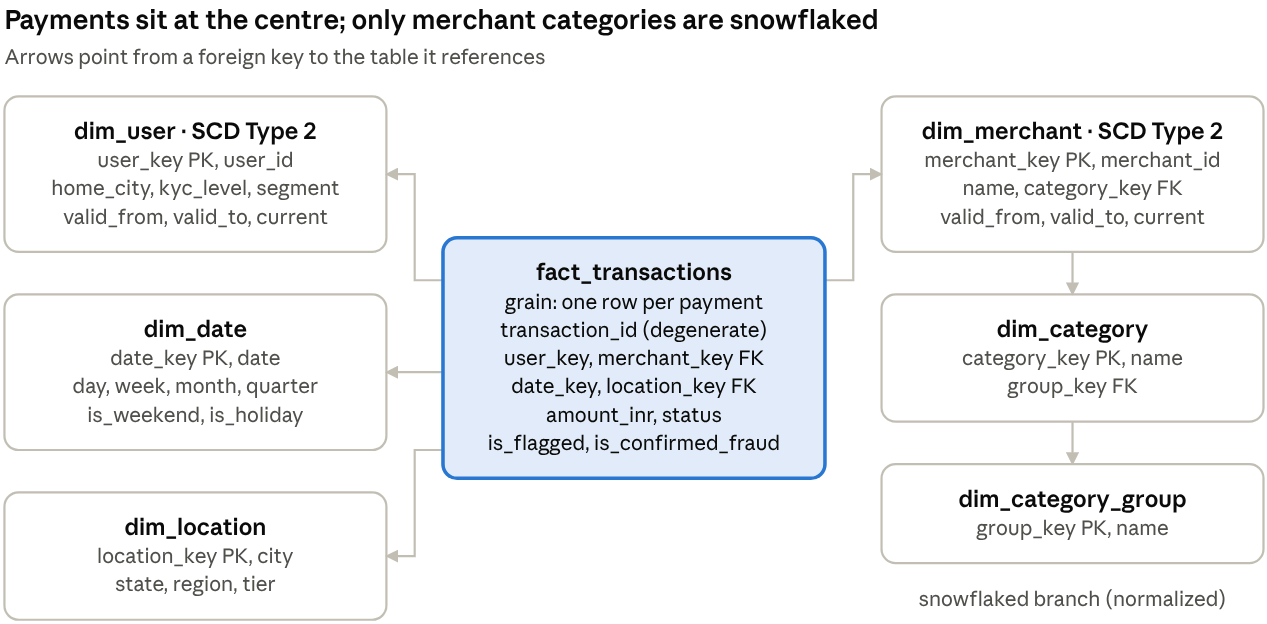
Layer	Where	What happens	Tool
Bronze	S3, Iceberg	Raw events exactly as received, partitioned by day	Kafka sink
Silver	S3, Iceberg	Typed, deduplicated by `transaction_id`, PII masked, bad rows to quarantine	Spark
Raw (warehouse)	Snowflake	Silver tables loaded as they are	Snowpipe / COPY
Staging	Snowflake	Renamed, cast, one model per source table	dbt
Marts	Snowflake	Star schema: facts and SCD Type 2 dimensions	dbt

Layer	Where	What happens	Tool
Consumption	Superset, console, API	Dashboards, case review, lookups	Superset, React, FastAPI

Every batch job processes one date partition and overwrites it on rerun. That makes reruns and backfills safe: running a day twice gives the same result as running it once.

Warehouse design

The marts use a star schema for speed and simplicity, with one snowflaked branch (merchant categories) to show both designs and benchmark the difference.



Warehouse marts · 1 central fact, 6 dimensions

Dimensions stay wide and denormalized except merchant categories, which are normalized because many merchants share a small, shared category tree.

Table	Type	Grain (one row per)	Notes
`fact_transactions`	Transaction fact	Payment	About 100M rows after backfill; clustered by `date_key`

Table	Type	Grain (one row per)	Notes
`fact_fraud_alerts`	Transaction fact	Fraud alert	From the Flink alerts topic; joined to case labels
`fact_case_labels`	Transaction fact	Analyst label on an alert	Written by the console; feeds rule precision
`fact_merchant_daily`	Periodic snapshot	Merchant per day	Revenue, payment count, fraud rate
`dim_user`	SCD Type 2	User version	New row when city, KYC level or segment changes
`dim_merchant`	SCD Type 2	Merchant version	New row when category or status changes
`dim_date`	Static	Calendar day	Includes Indian public holidays
`dim_location`	Type 1	City	Overwritten in place
`dim_category`, `dim_category_group`	Snowflaked	Category, category group	Normalized branch

Snowflake-specific tuning: Snowflake has no indexes, so performance comes from clustering keys on large facts, materialized views for heavy dashboards, and right-sized warehouses. Every tuning change is recorded as a before/after timing in the README.

dbt layers: `staging` (one model per source table, renamed and cast) → `intermediate` (joins, dedupe, SCD logic with snapshots) → `marts` (the tables above).

Data quality and validation

Bad data is checked at every layer, quarantined with its reasons, and blocks publishing when a critical check fails.

Layer	Check	Tool	On failure
Kafka	Record matches the registered Avro schema; compatibility mode `BACKWARD`	Schema Registry	Producer error; record never enters the topic
Consumers	Record can be parsed and processed	Flink / sink code	Sent to `payments.dlq` after retries
Silver	Amount above 0, valid UPI ID format, timestamp not in the future, known currency, no duplicate `transaction_id`	Great Expectations	Row moved to `quarantine` table with reasons
Warehouse	`unique`, `not_null`, `relationships`, `accepted_values`, row-count and range checks	dbt tests + dbt-expectations	Airflow stops the run before marts publish
Reconciliation	Row counts and rupee totals match across Bronze, Silver and Snowflake for each day	Airflow task (SQL)	Alert; marts stay on the previous day
Freshness	Latest event in marts is under the agreed delay	dbt source freshness	Alert to on-call
Fraud rules	Precision and recall against the generator's ground-truth labels	dbt model + console labels	Shown on the pipeline health screen

Security checks also run here: PII columns (phone number, UPI ID) are hashed or masked in Silver, so raw PII never reaches the warehouse marts.

Fraud Ops Console

The console is where analysts act; Superset is where everyone reads. The console stays small (four screens) and never rebuilds charts Superset already provides.

Screen	What the analyst does	Data source	Concepts shown
Live alerts	Watches new fraud alerts arrive, filters by rule and city	`GET /alerts/stream` (SSE)	Server-Sent Events, backpressure on the client
Case review	Opens an alert, sees the user's recent payments, marks it "confirmed fraud" or "false alarm"	`GET /cases/{id}`, `POST /cases/{id}/label`	Idempotent writes, optimistic locking, labels flow back to the warehouse
Pipeline health	Checks Airflow runs, data-quality results, Kafka consumer lag, rule precision	`GET /health/pipeline`	Observability, SLOs
Lookup	Searches a user or merchant and sees transactions and alerts	`GET /users/{id}`, `GET /merchants/{id}`	Pagination, Redis caching, indexes, no N+1 queries

Stack: React, TypeScript, Vite, TanStack Query, Tailwind CSS, Recharts (small charts only), Vitest for unit tests and Playwright for browser tests.

Security rules: login with short-lived JWT access tokens; the API allows only the console's origin (CORS); React escapes text by default and `dangerouslySetInnerHTML` is banned by lint; a Content-Security-Policy header is set by the Nginx container that serves the build.

Reliability patterns

Most of the system-design concepts reviewed are built properly, some appear in a light form, and eight are skipped on purpose. The table maps each built concept to where it lives.

Concept	Where it lives	Status
Message queues, pub/sub, event-driven architecture	Kafka topics and consumer groups	Built

Concept	Where it lives	Status
Idempotency	Dedupe by `transaction_id`; partition overwrite on rerun; idempotent label writes	Built
Dead letter queue	`payments.dlq` topic + quarantine table	Built
Backpressure	Flink credit-based flow control; demo by throttling the sink	Built
Exactly-once / avoiding distributed transactions	Flink two-phase-commit sinks instead of cross-database transactions	Built
Timeouts, retries, exponential backoff	API clients, Airflow task retries	Built
Circuit breaker	Calls to the refunds API	Built
Partitioning	Iceberg day partitions; Kafka partitions by `user_id` (the sharding idea)	Built
Indexing, query optimization, N+1	Postgres and MySQL indexes; Snowflake clustering keys and Query Profile; API joins	Built
Connection pooling	SQLAlchemy pool in the API	Built
Pessimistic locking, race conditions	`SELECT ... FOR UPDATE` on balances; concurrency tests	Built
Optimistic locking	Version column on cases and merchant records	Light
Eventual consistency	Warehouse lags the stream; freshness target documented	Built
Replication, leader election, CAP	Kafka replication and KRaft controller; trade-offs explained in this doc	Light
Memory leaks, garbage collection	Flink state TTL; Spark executor memory tuning	Built
Caching, cache invalidation	Redis user profiles cleared by CDC events	Light
Load balancing, reverse proxy, service discovery	Kubernetes Services, Traefik ingress, cluster DNS	Built

Concept	Where it lives	Status
API gateway, rate limiting	Traefik middleware	Light
Autoscaling, horizontal scaling	Horizontal Pod Autoscaler on the API, tested under load	Built
Rolling deploys, rollbacks	Kubernetes rolling updates, `helm rollback`, Iceberg time travel for data	Built
Health checks, liveness and readiness probes	Every service	Built
Feature flags, canary	New fraud rules run in shadow mode (log only) for a slice of users	Light
Saga	Refund flow with compensating steps	Optional
Cron jobs	Airflow schedules	Built

Skipped on purpose: blue-green deployment (doubles cost), read replicas (CDC already offloads reads), CDN and edge caching (no public website), WebSockets and long polling (alerts flow one way, so SSE fits), WAF (paid) and DDoS protection (AWS Shield Standard is automatic). CSRF needs only a light guard because the API uses bearer tokens, not cookies.

Security and privacy

Raw PII stays in the source systems and Bronze; everything downstream sees masked values, and every person or service gets only the access it needs.

Area	Control
Secrets	AWS Secrets Manager in the cloud, Kubernetes Secrets locally; nothing in Git; `gitleaks` scan in CI
Identity and access	Least-privilege IAM roles per service; Snowflake roles `LOADER`, `TRANSFORMER`, `ANALYST`; analysts cannot read PII columns
API authentication	Short-lived JWT access tokens; signing keys rotated, identified by `kid`

Area	Control
Encryption in transit	TLS on public endpoints, Snowflake and AWS APIs
Encryption at rest	S3 server-side encryption with KMS; Snowflake encrypts by default
PII handling	Phone and UPI ID hashed with a salt in Silver; masking policy in Snowflake for the rare column that must stay readable
Input safety	Parameterized SQL everywhere; Pydantic validation on every request; no raw string-built queries in dbt macros
Web safety	CORS limited to the console origin; React output escaping; Content-Security-Policy header
Audit	Every case label and data-access request logged with user and time
Supply chain	`pip-audit`, `npm audit` and `bandit` in CI; images built from slim, pinned base images

All data is synthetic. Sample data uses reserved domains (example.com) and clearly fake phone numbers so nothing resembles a real person.

Observability

Three SLOs define "working"; every alert maps to one of them. The targets below are starting points, to be adjusted after the first load tests.

SLI (what we measure)	SLO (target)	Measured by	Alert when
Fraud alert latency: payment event time to alert visible in the API	99% under 5 seconds	Flink metrics + API timestamps	Above target for 10 minutes
Mart freshness: age of the newest event in Snowflake marts	Under 26 hours	dbt source freshness	Any breach

SLI (what we measure)	SLO (target)	Measured by	Alert when
Daily pipeline success: Airflow runs completing with all critical checks green	99% of days per month	Airflow metrics	Any failed run

Error budget: the 1% allowed to fail. If a budget is spent, new features pause until reliability work brings it back.

Signals collected

- **Metrics** (Prometheus): Kafka consumer lag, events per second, Flink checkpoint duration and state size, API request rate, errors and latency, Spark job duration.
- **Logs**: structured JSON with a `run_id` or `trace_id` on every line.
- **Traces** (OpenTelemetry to Jaeger): API request to Kafka publish, and case label to warehouse.
- **Dashboards** (Grafana): one per SLO plus a platform overview.
- **Alerts** (Alertmanager): routed to email or a Telegram bot, both free.

CI/CD and infrastructure as code

No change reaches AWS without passing tests, and no AWS or Snowflake resource exists that is not described in Terraform.

Pipeline stages (GitHub Actions)

Trigger	Stage	What runs
Every push	Lint	`ruff`, `mypy`, `sqlfluff` for SQL, `eslint` for the console
Every push	Unit tests	`pytest` (Python, Spark local session), Flink job tests, Vitest
Every push	Security scan	`bandit`, `pip-audit`, `npm audit`, `gitleaks`
Pull request	Integration tests	Docker Compose stack in CI: generator to Kafka to sink to Postgres, end to end
Pull request	Data checks	`dbt build` (models + tests) against a CI Postgres schema

Trigger	Stage	What runs
Pull request	Infra plan	`terraform fmt`, `validate` and `plan`, posted as a PR comment
Merge to main	Build	Multi-stage Docker images, tagged with the Git SHA, pushed to Amazon ECR
Merge to main	Deploy	`terraform apply` (dev), then `helm upgrade` with a rolling update
Every night	Browser tests	Playwright against the deployed console

A failed health check after deploy triggers `helm rollback` to the previous release.

Terraform modules

Module	Creates
`network`	VPC, public subnet, security groups (no NAT gateway, to save credits)
`storage`	S3 buckets for Bronze, Silver, quarantine and Terraform state; lifecycle rules
`iam`	One role per service with least privilege
`compute`	EC2 instance running k3s; Kafka, Flink and Airflow run inside it
`registry`	ECR repositories
`secrets`	Secrets Manager entries
`snowflake`	Warehouses (auto-suspend 60 seconds), databases, roles, grants, storage integration with S3
`monitoring`	AWS Budgets alert and CloudWatch log groups

Terraform state lives in S3 with locking. Separate `dev` and `prod` workspaces keep environments apart.

Environments and cost plan

Build everything locally for free; open the cloud trials only for the final phase, because their clocks start at sign-up.

Environment	Runs on	Stands in for	Cost
Local	Docker Compose, then k3d (Kubernetes in Docker) on the laptop	Everything	Free
Local storage	MinIO	Amazon S3 (same API, no code change)	Free
Local warehouse	PostgreSQL schema `analytics`	Snowflake (dbt models run on both)	Free
Cloud	AWS free plan	S3, EC2, ECR, IAM, Secrets Manager, CloudWatch	Credits
Cloud warehouse	Snowflake trial in an AWS region	Production warehouse	Trial credits
Optional	Redshift Serverless trial	Second warehouse target for the same dbt project	Trial credits

Trial facts to plan around

- AWS: new customers get $100 in credits at sign-up and can earn up to $100 more; the free plan ends after 6 months or when credits run out, whichever comes first (AWS). Some enterprise-oriented services are not available on the free plan (AWS News Blog).
- Snowflake: 30 days and $400 of usage, no billing information required (Snowflake).
- Redshift Serverless: a $300 credit usable within 90 days for accounts that have never used it; separate from the AWS Free Tier (AWS).

Cost rules

1. Create an AWS Budgets alert on day one.
2. Stop the EC2 instance whenever no demo or test is running.
3. No NAT gateway, managed EKS or managed Kafka (MSK); they burn credits fast.
4. Snowflake warehouses use size X-Small with auto-suspend after 60 seconds.
5. `terraform destroy` after each demo; `terraform apply` rebuilds in minutes.

6. Record a demo video and screenshots before the Snowflake trial ends; the repo stays runnable on Postgres afterwards.

Laptop requirement: running Kafka, Flink, Spark, Airflow, Postgres and MySQL together needs about 16 GB of RAM. With 8 GB, run one phase's services at a time.

Repository layout

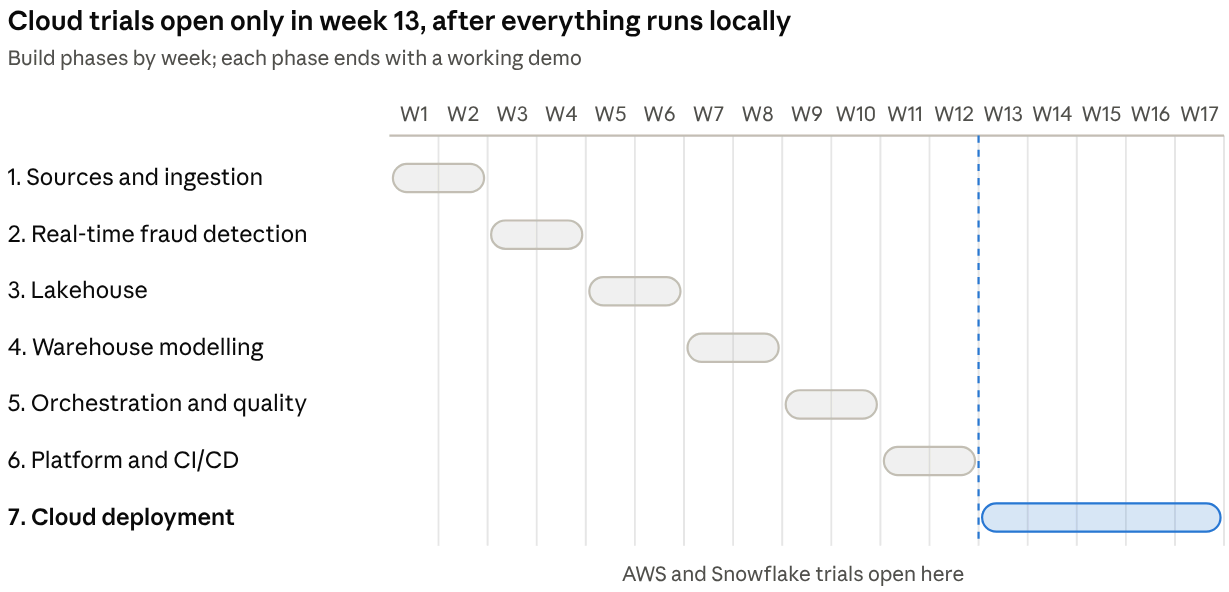
One monorepo, one folder per deployable part, so CI can test and build each part on its own.

```
paystream/
├── generator/          # Python transaction generator (Avro producer)
├── sources/
│   ├── payments-core/  # Postgres schema + seed data
│   ├── merchant-app/   # FastAPI + MySQL merchant onboarding app
│   └── refunds-api/    # FastAPI refunds service
├── ingestion/
│   ├── debezium/       # CDC connector configs
│   └── schemas/        # Avro schemas (single source of truth)
├── streaming/          # Flink jobs: fraud rules, merchant metrics
├── batch/              # Spark jobs: bronze -> silver, compaction, backfill
├── warehouse/          # dbt project: staging, intermediate, marts, tests
├── orchestration/      # Airflow DAGs
├── quality/            # Great Expectations suites
├── api/                # FastAPI alerts, cases, lookup, SSE
├── console/            # React + TypeScript Fraud Ops Console
├── infra/
│   ├── terraform/      # modules + dev/prod environments
│   └── helm/           # charts for every service
├── observability/      # Prometheus rules, Grafana dashboards
├── docs/               # this design doc, ADRs, runbooks
├── .github/workflows/  # CI/CD pipelines
├── docker-compose.yml  # local stack
└── Makefile            # make up | test | load | down
```

The Customer Data Integration Pipeline stays in its own repository. PayStream reads its Postgres tables through CDC, the same way it reads any other internal application.

Key decisions are recorded as short Architecture Decision Records in `docs/adr/` (for example: "Snowflake as the main warehouse", "rules before ML for fraud").

Roadmap



Build roadmap · 7 phases over 17 weeks

Six two-week phases run entirely on the laptop for free; the final five weeks use the cloud trials. The console is built alongside the backend: Live alerts in Phase 2, Pipeline health in Phase 5, Case review and Lookup in Phase 6. Every task is listed in the Build checklist below.

Build checklist

Each phase ends with a working demo. Tick tasks as they finish; a phase is done only when its demo runs from `make up`.

Phase 1: Sources and ingestion (weeks 1–2)

- [ ] Create the monorepo, README skeleton, Makefile and `docker-compose.yml`
- [ ] Write Avro schemas for payment, user and merchant events
- [ ] Build the transaction generator with configurable rate and fraud injection
- [ ] Set up Postgres payments core and the MySQL merchant app (FastAPI)
- [ ] Run Kafka (KRaft) + Schema Registry + Kafka UI in Docker
- [ ] Configure Debezium CDC for Postgres and MySQL
- [ ] Write unit tests for the generator
- [ ] Demo: events and CDC changes visible in Kafka UI

Phase 2: Real-time fraud detection (weeks 3–4)

- [ ] Flink job: event time, watermarks, enrichment with CDC state
- [ ] Fraud rules: velocity, impossible travel, amount spike, with state TTL
- [ ] `fraud.alerts` topic and `payments.dlq` dead letter topic
- [ ] FastAPI alerts API with SSE stream
- [ ] Console: project setup and Live alerts screen
- [ ] Measure alert latency and precision/recall against ground truth
- [ ] Demo: injected fraud appears on screen within seconds

Phase 3: Lakehouse (weeks 5–6)

- [ ] MinIO + Iceberg catalog
- [ ] Bronze sink from Kafka, partitioned by day
- [ ] Spark job Bronze to Silver: types, dedupe, PII hashing
- [ ] Quarantine table for rejected rows
- [ ] Generate and backfill one year of history (about 100M rows)
- [ ] Spark tuning: partitioning, broadcast joins, compaction; record before/after times
- [ ] Demo: query Silver tables and roll back a bad load with time travel

Phase 4: Warehouse modelling (weeks 7–8)

- [ ] dbt project on Postgres `analytics` schema
- [ ] Staging models, then the star schema: facts, SCD Type 2 dimensions, snowflaked category branch
- [ ] dbt tests and dbt-expectations checks
- [ ] Write `requirements.md`: five stakeholder questions and their metric definitions
- [ ] Publish dbt docs (lineage graph) on GitHub Pages
- [ ] Demo: marts answer all five stakeholder questions

Phase 5: Orchestration and quality (weeks 9–10)

- [ ] Airflow DAG: Spark, load, `dbt build`, publish, with retries and SLAs
- [ ] Great Expectations suites on Silver
- [ ] Reconciliation task: counts and totals across layers
- [ ] Refunds API ingestion with timeouts, backoff and a circuit breaker

- [ ] Read the Customer Data Integration Pipeline's tables through CDC into `dim_user`
- [ ] Console: Pipeline health screen
- [ ] Demo: a bad file is quarantined and the run stops before publishing

Phase 6: Platform, CI/CD and observability (weeks 11–12)

- [ ] Multi-stage Dockerfiles; Helm charts; run all on k3d
- [ ] Liveness/readiness probes, HPA on the API, Traefik ingress with rate limits
- [ ] Prometheus, Grafana dashboards for the three SLOs, Alertmanager, OpenTelemetry traces
- [ ] GitHub Actions: lint, tests, security scans, integration stack, `dbt build`
- [ ] Console: Case review and Lookup screens; Vitest and Playwright tests
- [ ] Load test the API and the stream; record throughput
- [ ] Demo: kill a pod and a Kafka broker; system recovers and SLO dashboards show it

Phase 7: Cloud deployment and polish (weeks 13–17)

- [ ] Open the AWS account (free plan) and set a Budgets alert on day one
- [ ] Terraform modules: network, storage, IAM, compute, registry, secrets, monitoring
- [ ] Deploy k3s on EC2 with Helm; images from ECR
- [ ] Open the Snowflake trial; Terraform the Snowflake module; Snowpipe from S3
- [ ] Run the dbt project on Snowflake; tune with clustering keys and Query Profile
- [ ] Superset dashboards on Snowflake
- [ ] CI deploy on merge; rollback tested
- [ ] README: architecture, metrics table, cost per day, how to run
- [ ] Record a 5-minute demo video before the Snowflake trial ends
- [ ] `terraform destroy`

Risks and open questions

The biggest risk is not technical: it is finishing. Each phase is designed to be a complete demo on its own.

Risk	Impact	Mitigation
Scope too large for one person	Half-finished project	Vertical slices; each phase demo-able; cut optional items (saga, Redshift) first
"Tool soup" in interviews	Looks like buzzword stacking	Every tool has a one-line reason in the Components table and an ADR
Not understanding generated code	Fails deep interview questions	Write code with line-by-line explanation; end each phase by explaining it back
Laptop RAM below 16 GB	Stack will not run all at once	Run services per phase; use smaller Kafka and Flink memory settings
Cloud trial clocks run out	Lost demo environment	Start trials only in Phase 7; record video and screenshots; keep Postgres target working
Unexpected AWS charges	Real money spent	Free plan account, Budgets alert, stop EC2 when idle, `terraform destroy`
Synthetic data questioned	Seen as less realistic	Explain: real payment data is private; known ground truth enables precision and recall

Open questions

- How much RAM does the development laptop have? This decides whether the full stack runs at once.
- Which AWS region for S3, EC2 and the Snowflake account? Same region avoids transfer costs; check that Snowflake trials offer it.
- Is Redshift Serverless available on the AWS free plan, or only on the paid plan?
- Which ML extension, if any, after the rules: a gradient-boosted scorer served from Flink, or none?

Sources

- AWS Free Tier: $200 in credits and 6-month free plan (announcement)

- AWS News Blog: Free Tier update
- AWS Free Tier FAQs
- Snowflake free trial
- Snowflake trial accounts documentation
- Amazon Redshift Serverless free trial